

How to add a board to QEMU

Contents

QEMU NXP S32K3x8 board emulation – Group 9.....	2
Setting up the environment	2
Step 1 - Setting up the environment	2
Step 2 – Clone QEMU source code.....	2
Step 3 – Modding QEMU config file.....	2
Step 4 – Configure QEMU	3
S32k3x8.c	4
Memory Mapping	4
Memory map in QEMU	5
CPU initialization	5
Appendix A	7
Appendix B	8

QEMU NXP S32K3x8 board emulation – Group 9

This project aims to develop a new version of QEMU that support the NXP S32K3x8 board. QEMU has to emulate the cortex-m7, memory mapping and some peripheral. In our project we should be able to emulate UART and ADC.

Setting up the environment

Step 1 - Setting up the environment

First of all, we should ensure that all this package are installed

```
sudo apt update

sudo apt install -y git build-essential ninja-build pkg-config \
  libglib2.0-dev libpixman-1-dev zlib1g-dev \
  libfdt-dev libssl-dev \
  python3 python3-pip python3-venv \
  libaio-dev libcap-ng-dev libcurl4-gnutls-dev \
```

Step 2 – Clone QEMU source code

First of all, we should clone the git repository of QEMU:

```
git clone https://gitlab.com/qemu-project/qemu.git

cd qemu
```

Step 3 – Modding QEMU config file

QEMU uses Meson and Ninja for its build system. Inside qemu/ there are various files and directory. To add an architecture, we should go in the folder qemu/hw/arm/, inside this folder there are different architectures, so we create the file s32k3x8.c, that represent our board. As said QEMU uses Meson, so, to make sure that QEMU sees our board, we should modify qemu/hw/arm/meson.build:

```
arm_ss.add(when: 'CONFIG_S32K3', if_true: files('s32k3.c'))
```

Then inside the file qemu/hw/arm/Kconfig we should add:

```
config S32K3
bool
default y
depends on ARM
```

Step 4 – Configure QEMU

Now we should configure QEMU, so that when we compile, it reads correctly our board. To do so, in qemu/:

```
mkdir build
cd build

../configure --target-list=arm-softmmu
```

In the configuration, we first created the folder build, where we will have our instance of QEMU modded, and we will compile only arm architecture.

Then to reload the configuration, when some files are added (for example the peripherals):

```
meson setup build --reconfigure
```

Then to compile QEMU and test it, when we are in qemu/ type:

```
ninja -C build
./build/qemu-system-arm -M s32k3x8 -m 128M -nographic
```

S32k3x8.c

The environment is ready, but now we should write our board.

To set up the board, first of all we should write a MACRO defined by QEMU, which generates some code useful to initialize the board:

```
DEFINE_MACHINE("s32k3x8", s32k3x8_machine_init)
```

Where s32k3x8_machine_init is a function that we should write:

```
static void s32k3x8_machine_init(MachineClass *mc){
    mc->desc = "NXP S32K3x8 (Cortex-M7)";
    mc->init = s32k3x8_init;
    mc->default_ram_size = 128*1024*1024;
    mc->default_cpu_type = ARM_CPU_TYPE_NAME("cortex-m7");
}
```

And again, a function s32k3x8_init is needed. This function will initialize in our CPU, memories and peripherals.

Memory Mapping

In the documentation it is possible to find all of memory addresses for the S32k3x8 boards, in the table below will be listed all the memories and peripherals that will be mapped:

START ADDRESS [HEX]	MAX SIZE [KB]	Description
Memories		
0 x 0000 0000	64	ITCM (the real division depends on x version of boards)
0 x 0040 0000	2048	Program flash (block 0)
0 x 0060 0000	2048	Program flash (block 1)
0 x 0080 0000	2048	Program flash (block 2)
0 x 00A0 0000	2048	Program flash (block 3)
0 x 1000 0000	256	Data flash
0 x 2000 0000	128	DTCM (the real division depends on x version of boards)
0 x 2040 0000	256	SRAM0
0 x 2044 0000	256	SRAM1
0 x 2048 0000	256	SRAM2
Peripherals		
0 x 4032 8000	16	Low Power UART 0 (LPUART0)
0 x 400A 0000	16	ADC 0

Memory map in QEMU

In QEMU to instantiate a memory, first we need to declare the MemoryRegion a structure dedicated. First of all, it is needed to declare the global system memory:

```
MemoryRegion *system_memory = get_system_memory();
```

Now having the global system memory, we should declare the various memory regions and attach them to the system_memory. So, to create a memory region:

```
MemoryRegion *mem_reg = g_new(MemoryRegion, 1);  
//memory_region_init_ram or rom  
memory_region_init_ram(mem_reg, NULL, "s32k3x8.mem_reg", mem_reg_BLOCK_SIZE, &err);  
memory_region_add_subregion(system_memory, mem_reg_BASE_ADDRESS, mem_reg);
```

To create memory region such SRAM0, SRAM1, SRAM2, we need to create a SRAM memory_region, then declare SRAM0, SRAM1, SRAM2 memory_region, and to them initialize the memory of each of them (no init of SRAM). Then attach them to SRAM with the correctly offset, and at the end attach the SRAM memory region to system_memory.

CPU initialization

To initialize the CPU, we need to declare armv7m:

```
DeviceState *armv7m;  
armv7m = qdev_new(TYPE_ARMV7M);
```

Then we need to link the memory to the CPU:

```
object_property_set_link(OBJECT(armv7m), "memory", OBJECT(get_system_memory()), &err);
```

Then some settings are needed:

```
qdev_prop_set_uint32(armv7m, "num-irq", 240);  
qdev_prop_set_string(armv7m, "cpu-type", machine->cpu_type);  
qdev_prop_set_bit(armv7m, "enable-bitband", true);  
  
cpuclock = clock_new(OBJECT(machine), "cpuclock");  
clock_set_hz(cpuclock, 48 * 1000 * 1000);  
qdev_connect_clock_in(armv7m, "cpuclock", cpuclock);
```

Then we need to initialize the bus system:

```
if (!sysbus_realize(SYS_BUS_DEVICE(armv7m), &err)) {  
    error_reportf_err(err, "Could not realize ARMv7M device: ");  
    exit(1);  
}
```

To load a kernel image is needed to write:

```
if(machine->kernel_filename)
    armv7m_load_kernel(ARMV7M(armv7m)->cpu, machine->kernel_filename, 0,
        ITCM_BLOCK_SIZE);
```

Appendix A

Needed library to import:

- qemu/osdep.h
- qapi/error.h
- hw/sysbus.h
- hw/arm/boot.h
- net/net.h
- hw/boards.h
- exec/address-spaces.h
- system/system.h
- system/reset.h
- hw/arm/armv7m.h
- hw/qdev-clock.h
- qom/object.h
- qobject/qlist.h

Appendix B